[Nam Le] Threads & Synchronization

0. Overview Thread

Thread is **operating system concept**
Pthread is **API standard**

0.1 Thread

Thread is smallest execution unit inside Process

- Threads in a process will share virtuall address space (text, data/bss, heap). Same PID
- Each thread has their Stack memory, with ThreadID + signal mask + CPU register context + **errno**
- Pros (vs Process)
  - Tranfer data is faster than Process (because only write/read global variable or heap memory)
  - Create thread fast : kernel does not need to create page tables or allocate memory again
- Cons : use shared memory with others thread, if 1 thread error then others can shutdown or interupt immediately then terminate process

0.2 Pthread (user-space API Standard)

They are library/C function, use **<pthread.h>**, support development create and synchonize on LINUX/unix
Now go thread Pthread (POSIX thread)

1. POSIX Thread - Race Condition - Mutex

1.1 Thread concept

Like processes, threads are a mechanism that permits an application to perform **multiple tasks** concurrently
A single process can contain multiple threads

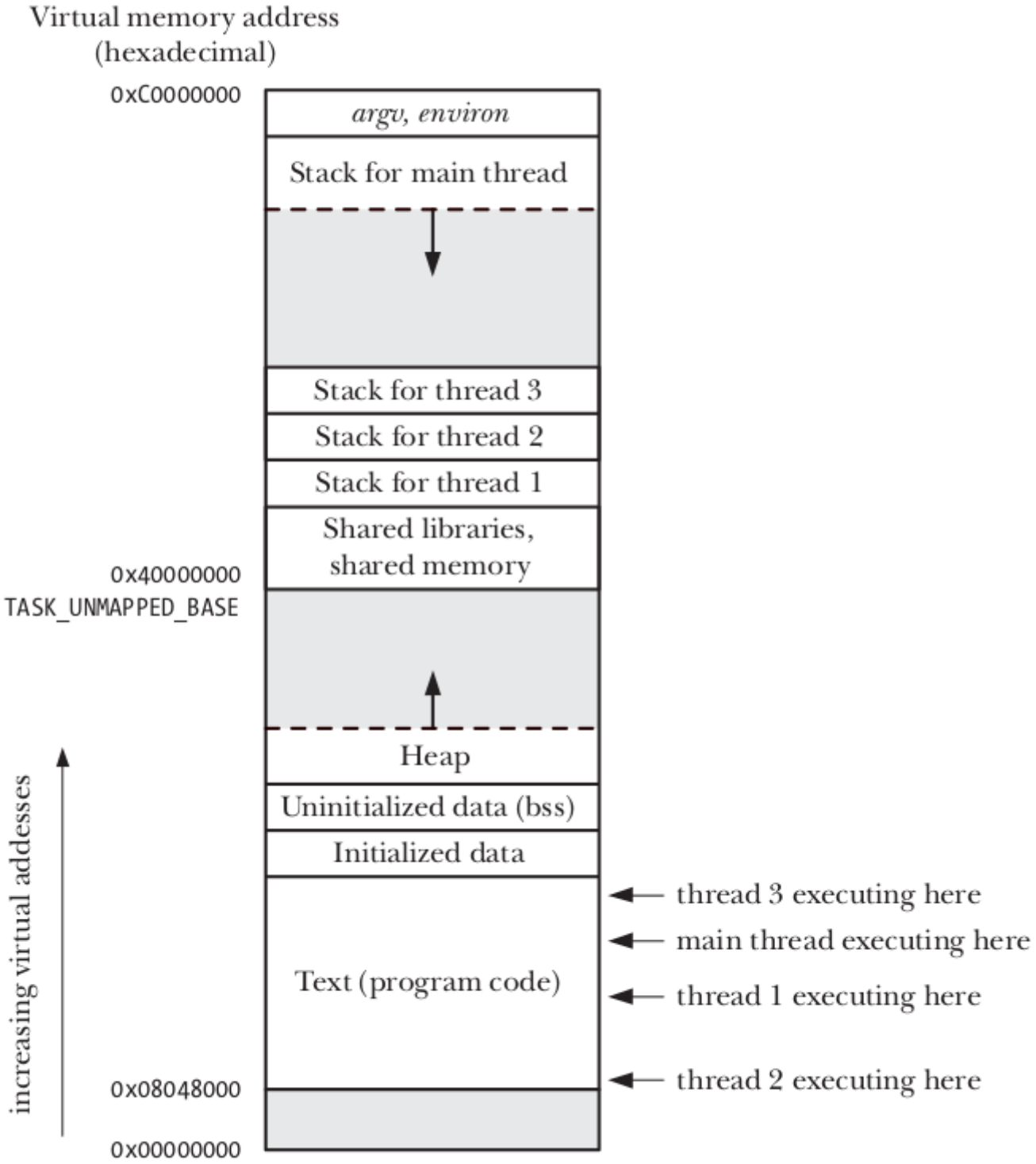


Figure 29-1: Four threads executing in a process (Linux/x86-32)

All of these threads are **independently** executing the same program, and they all **share** the same **global memory**, including the **initialized data, uninitialized data, and heap segments**

The distinction of each thead:

- thread ID ;
- signal mask;
- thread-specific data ;
- alternate signal stack (*sigaltstack()*);
- the *errno* variable;
- floating-point environment (see *fenv(3)*);
- realtime scheduling policy and priority ;
- CPU affinity;
- capabilities;
- stack (local variables and function call linkage information).

PThreads API

Pthreads data types:

Data type	Description
pthread_t	Thread identifier
pthread_mutex_t	Mutex
pthread_mutexattr_t	Mutex attributes object
pthread_cond_t	Condition variable
pthread_condattr_t	Condition variable attributes object
pthread_key_t	Key for thread-specific data
pthread_once_t	One-time initialization control context
pthread_attr_t	Thread attributes object

Thread and *errno*

In the traditional UNIX API, errno is a **global integer** variable. However, this doesn't suffice for threaded programs(where they share virtual mem, static data and heap segment)

- If a thread made a function call that returned an error in a global errno variable
- this would confuse other threads that might also be making function calls and checking errno --> race condition

--> Therefore, in threaded programs, each thread has its own errno value - **[MACRO]**

Return value from Pthreads functions

All Pthreads functions return 0 on success or a positive value on failure
The failure value is one of the same values that can be placed in errno by traditional UNIX system calls.

Compiling Pthreads programs

On Linux, programs that use the Pthreads API must be compiled with the **cc –pthread** option

1.2 Thread Creation

The *pthread_create()* function creates a new thread.
Syntax:

```
#include <pthread.h>
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,void *(*start)(void *), void
*arg);
                                        Returns 0 on success, or a positive error number on
error
```

Example:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void* thread_func(void* arg) {
    int thread_id = *((int*) arg);

    printf("[Thread %d] is running... (tid = %lu)\n", thread_id, pthread_self());
    sleep(1); // Assump handling work
    printf("[Thread %d] completely!\n", thread_id);

    pthread_exit(NULL);
}
int main() {
    pthread_t threads[3];
    int thread_ids[3] = {1, 2, 3};

    printf("[Main] Start create 3 threads\n");

    for (int i = 0; i < 3; i++) {
        int ret = pthread_create(&threads[i], NULL, thread_func, (void*)&thread_ids[i]);
        if (ret != 0) {
            perror("Fail to create thread");
            return 1;
```

```
        }
        printf("[Main] Thread %d is created\n", thread_ids[i]);
    }
    /*Join 3 threads together, wait for sucess of all 3 threads*/
    for (int i = 0;  i< 3; i++) {
        pthread_join(threads[i], NULL);
        printf("[Main] Joined Thread %d\n", thread_ids[i]);
    }
    return 0;
}
```

Run: `gcc thread_create.c -o thread_create -pthread`. Then `.thread_create`
Result:

```
[Main] Start create 3 threads
[Main] Thread 1 is created
[Thread 1] is running... (tid = 128621131855424)
[Main] Thread 2 is created
[Thread 2] is running... (tid = 128621123462720)
[Thread 3] is running... (tid = 128621115070016)
[Main] Thread 3 is created
[Thread 1] completely!
[Thread 2] completely!
[Thread 3] completely!
[Main] Joined Thread 1
[Main] Joined Thread 2
[Main] Joined Thread 3
```

NOTE:

- `pthread_create(&threads[i], NULL, thread_func, (void*)&thread_ids[i]);`: create new thread, assign poniter thread_ids as argument for function run on this thread.
- The print number of `[Thread 1/2/3] is running...` is not deterministic. **THIS IS MULTI_THREAD**
- `pthread_join` in main() will block until each thread comletely run. If not join, main() can end before thread completely run
- use thread_ids[] array instead of use directly i value to avoid race condition when variable of loop change before thread read

SUMMARY we have 4 arguments for function:

- `thread`: point to buffer `pthread_t` type. **New thread can be schedule and run before `pthread_create()` complete.** We can use `pthread_self()` to get **thread ID**
- `attr`: a pointer to a *pthread_attr_t* object (size and position of Stack, schedule policy, detached state)
- `start`: the executable function, which execute new thread
- `arg`: argument is transmited to function

1.3 Thread termination

The *pthread_exit()* function terminates the calling thread, and specifies a return value that can be obtained in another thread by calling *pthread_join()*. Syntax:

```
include <pthread.h>
void pthread_exit(void *retval);
```

Calling *pthread_exit()* is equivalent to performing a return in the thread's start function, with the difference that `pthread_exit()` can be called from any function that has been called by the thread's start function

If the main thread calls `pthread_exit()` instead of calling `exit()` or performing a return, then the other threads **continue to execute**.

1.4 Race condition

Lab 1

Example:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
int counter = 0;
void* worker(void* arg) {
    int thread_id = *((int*) arg);

    printf("[Thread %d] is running... (tid = %lu)\n", thread_id, pthread_self());
    for (int i = 0; i < 1000000; i++) {
        counter++;
    }
```

```
    printf("[Thread %d] completely!\n", thread_id);

    pthread_exit(NULL);
}
int main() {
    pthread_t threads[4];
    int thread_ids[4] = {1, 2, 3, 4};

    printf("[Main] Start create 3 threads\n");

    for (int i = 0; i < 4; i++) {
        int ret = pthread_create(&threads[i], NULL, worker, (void*)&thread_ids[i]);
        if (ret != 0) {
            perror("Fail to create thread");
            return 1;
        }
        printf("[Main] Thread %d is created\n", thread_ids[i]);
    }
    /*Join 4 threads together, wait for sucess of all 4 threads*/
    for (int i = 0;  i < 4; i++) {
        pthread_join(threads[i], NULL);
        printf("[Main] Joined Thread %d\n", thread_ids[i]);
    }
    printf("After 4 threads handle successfully, Counter = %d\n", counter);
    return 0;
}
```

Result:

```
[Main] Start create 3 threads
[Main] Thread 1 is created
[Thread 1] is running... (tid = 132063560726080)
[Main] Thread 2 is created
[Main] Thread 3 is created
[Main] Thread 4 is created
[Thread 2] is running... (tid = 132063552333376)
[Thread 3] is running... (tid = 132063543940672)
[Thread 4] is running... (tid = 132063535547968)
[Thread 2] completely!
[Thread 1] completely!
```

```
[Main] Joined Thread 1
[Main] Joined Thread 2
[Thread 4] completely!
[Thread 3] completely!
[Main] Joined Thread 3
[Main] Joined Thread 4
After 4 threads handle successfully, Counter = 1359081
```

--> We can see that Counter final = 1359081, but expected result is 4.000.000 because each thread increase 1.000.000 counter --> **This is race condition** in thread.

Explain this sistuation:

When 4 threads run simultaneously, CPU scheduler can interrupt Thread 1 right after load data, assump old value = `2000`
Thread 2 come to run, increase the counter to `3000` and write on **RAM**
While Thread 1 is permitted to continue run, it use old value `2000` is loaded before, increase to `2001` and overwrite on **RAM**
--> Wrong result

1.4 Mutex

1.4.1 Definition

Mutex is a *mechanism* to ensure that only one thread at a time can access the variable
More generally, mutexes can be used to ensure atomic access to any shared resource, but protecting shared variables is the most common use. Mutex has 2 states: *locked* and *unlocked*

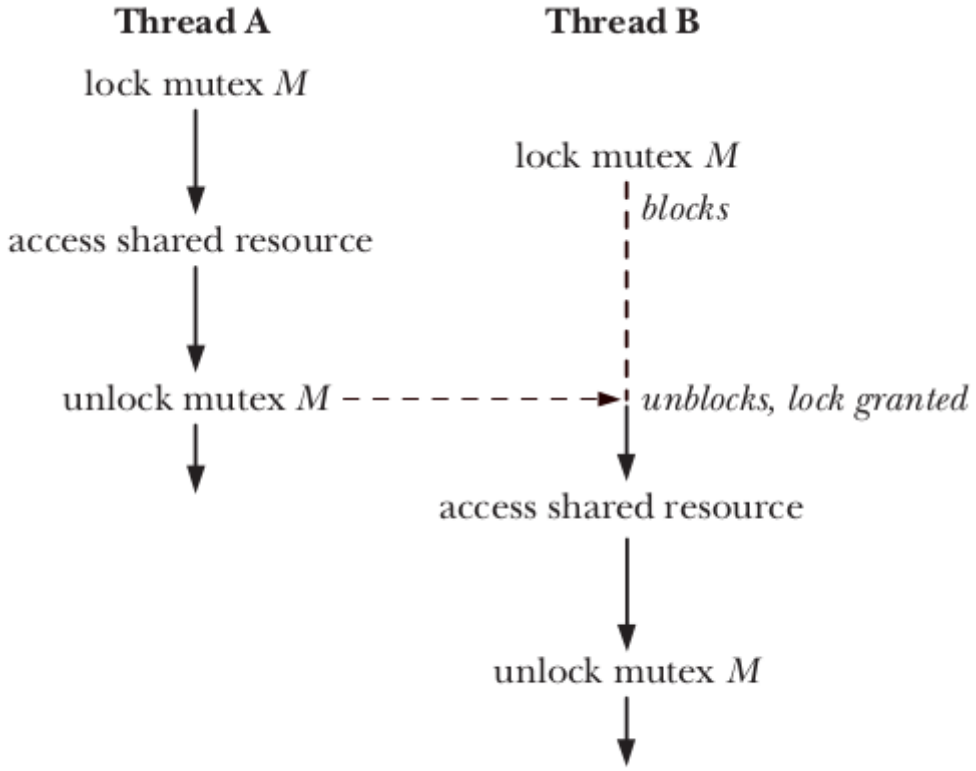


Figure 30-2: Using a mutex to protect a critical section

Protocol for accesing a resource:

- lock the mutex for the shared resource;
- access the shared resource; and
- unlock the mutex.

Critical section: Multiple threads try to execute this block of code
but the fact that only one thread can hold the mutex (the others remain blocked) means that only one thread at a time can enter the block.\

1.4.2 Example

A mutex is a variable of the type *pthread_mutex_t*. Before it can be used, a mutex must always be initialized.
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;

Lab 2

Example will fix race condition above

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
int counter = 0;
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;      /*
void* worker(void* arg) {
    int thread_id = *((int*) arg);

    printf("[Thread %d] is running... (tid = %lu)\n", thread_id, pthread_self());
    for (int i = 0; i < 1000000; i++) {
        int s = pthread_mutex_lock(&mtx);          /* add mutex lock */
        counter++;
        s = pthread_mutex_unlock(&mtx);            /* unlock mutex */
    }
    printf("[Thread %d] completely!\n", thread_id);

    pthread_exit(NULL);
}
int main() {
    pthread_t threads[4];
    int thread_ids[4] = {1, 2, 3, 4};

    printf("[Main] Start create 3 threads\n");

    for (int i = 0; i < 4; i++) {
        int ret = pthread_create(&threads[i], NULL, worker, (void*)&thread_ids[i]);
        if (ret != 0) {
            perror("Fail to create thread");
            return 1;
        }
        printf("[Main] Thread %d is created\n", thread_ids[i]);
    }
    /*Join 4 threads together, wait for sucess of all 4 threads*/
    for (int i = 0;  i < 4; i++) {
        pthread_join(threads[i], NULL);
        printf("[Main] Joined Thread %d\n", thread_ids[i]);
    }
    printf("After 4 threads handle successfully, Counter = %d\n", counter);
    return 0;
}
```

Result:

```
[Main] Start create 3 threads
[Main] Thread 1 is created
[Thread 1] is running... (tid = 135842274211392)
[Main] Thread 2 is created
[Main] Thread 3 is created
[Thread 2] is running... (tid = 135842265818688)
[Main] Thread 4 is created
[Thread 3] is running... (tid = 135842257425984)
[Thread 4] is running... (tid = 135842249033280)
[Thread 3] completely!
[Thread 2] completely!
[Thread 1] completely!
[Main] Joined Thread 1
[Main] Joined Thread 2
[Main] Joined Thread 3
[Thread 4] completely!
[Main] Joined Thread 4
After 4 threads handle successfully, Counter = 4000000
```

2. Condition Variable - Producer/Consumer - Semaphore

2.1 Condition Variable

Example demonstate *condition variable_* is useful

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
```

```
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
static int avail = 0;
int stop_producer = 0;

#define TOTAL_UNITS 5

/* Code to produce a unit omitted */
void* producer(void *arg) {
    for (int i = 1; i <= TOTAL_UNITS; i++) {
        sleep(1); /* assump produce time is 1s*/

        int s = pthread_mutex_lock(&mtx);
        avail++;                                    /* Let consumer know another unit is
available */
        printf("[producer] produced 1 unit, avail = %d\n", avail);
        s = pthread_mutex_unlock(&mtx);
    }
    pthread_mutex_lock(&mtx);
    stop_producer = 1;
    pthread_mutex_unlock(&mtx);

    printf("[Producer] Finished work\n");
    pthread_exit(NULL);
}
/* in main (consumer) thread */
int main() {
    pthread_t prod;
    int units_consumed = 0;
    pthread_create(&prod, NULL, producer, NULL);

    for (;;) {
    pthread_mutex_lock(&mtx);

    while (avail > 0) {                    /* Consume all available units */
        avail--;
        units_consumed++;
        printf("[Consumer] Consumed 1 unit. Exist: %d (Total consume: %d)\n", avail,
units_consumed);
    }
```

```
    /* Stop condition */
    if (stop_producer && avail == 0) {
            pthread_mutex_unlock(&mtx);
            break;
        }
    pthread_mutex_unlock(&mtx);
    }

    pthread_join(prod, NULL);
    pthread_mutex_destroy(&mtx);

    printf("[Main/Consumer]End. Total consumer: %d/%d\n", units_consumed, TOTAL_UNITS);
    return 0;
}
```

We can see that producer always increase *avail* variable while consumer want to decrease it --> comsumer continously run for loop to `lock` - `check` - `unlock` --> It waste CPU time
Affect:

```
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  29304 hoainam+  20   0   76512   1604   1492 R 100,0   0,0   0:04.91
condition_varia
```

A *condition variable* will remedies this problem. It allows a thread to sleep (wait) until another thread notifies (signals) it that it must do something

A condition variable has the type *pthread_cond_t* --> **`pthread_cond_t cond = PTHREAD_COND_INITIALIZER;`**

To fix this problem, we have:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
static int avail = 0;
int stop_producer = 0;

#define TOTAL_UNITS 5
```

```
/* Code to produce a unit omitted */
void* producer(void *arg) {
    for (int i = 1; i <= TOTAL_UNITS; i++) {
        sleep(1); /* assump produce time is 1s*/

        pthread_mutex_lock(&mtx);
        avail++;                                    /* Let consumer know another unit is
available */
        printf("[producer] produced 1 unit, avail = %d\n", avail);

        pthread_cond_signal(&cond); /*Wake up consumer who is waiting for unit*/
        pthread_mutex_unlock(&mtx);
    }
    pthread_mutex_lock(&mtx);
    stop_producer = 1;

    pthread_cond_signal(&cond); /*Wake up Consumer check stop condition*/
    pthread_mutex_unlock(&mtx);

    printf("[Producer] Finished work\n");
    pthread_exit(NULL);
}
/* in main (consumer) thread */
int main() {
    pthread_t prod;
    int units_consumed = 0;
    pthread_create(&prod, NULL, producer, NULL);

    for (;;) {
    pthread_mutex_lock(&mtx);
    while (avail == 0 && !stop_producer ) {
        pthread_cond_wait(&cond, &mtx);
    }

    while (avail > 0) {                    /* Consume all available units */
        avail--;
        units_consumed++;
        printf("[Consumer] Consumed 1 unit. Exist: %d (Total consume: %d)\n", avail,
units_consumed);
    }
```

```
    /* Stop condition */
    if (stop_producer && avail == 0) {
            pthread_mutex_unlock(&mtx);
            break;
        }
    pthread_mutex_unlock(&mtx);
    }

    pthread_join(prod, NULL);
    pthread_mutex_destroy(&mtx);

    printf("[Main/Consumer]End. Total consumer: %d/%d\n", units_consumed, TOTAL_UNITS);
    return 0;
}
```

---> Result: **Consumer thread** go efficiently to **sleep state** and completely **free CPU**

NOTE:

- A condition variable is always used in conjunction with a mutex
- the pthread_cond_wait() call is placed within a while loop rather than an if statement
- Signal transmission mechanism:
  - **pthread_cond_signal()**: Wake up **at least** 1 thread is waiting on _condition variable _ --> Should use when all threads are handle the same work
  - **pthread_cond_broadcast()**: Wake up **all waiting thread**. Force use when threads handle different tasks with different logic conditions
- 2 ways to efficiently handle between Signal and Unlock mutex
  - (Signal first, Unlock later): Lock Mutex --> Increase *avail* --> Transmit **pthread_cond_signal()** --> Unlock **pthread_mutex_unlock()**
  - (Unlock first, Signal later): Lock Mutex --> Increase *avail* --> Unlock **pthread_mutex_unlock()**--> Transmit **pthread_cond_signal()**

2.2 Producer/Consumer

Lab 4

Example:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
```

```
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
static int avail = 0;
int stop_producer = 0;

#define TOTAL_UNITS 5

/* Code to produce a unit omitted */
void* producer(void *arg) {
    for (int i = 1; i <= TOTAL_UNITS; i++) {
        sleep(1); /* assump produce time is 1s*/

        pthread_mutex_lock(&mtx);
        avail++;                                    /* Let consumer know another unit is
available */
        printf("[producer] produced 1 unit, avail = %d\n", avail);

        pthread_cond_signal(&cond); /*Wake up consumer who is waiting for unit*/
        pthread_mutex_unlock(&mtx);
    }
    pthread_mutex_lock(&mtx);
    stop_producer = 1;

    pthread_cond_signal(&cond); /*Wake up Consumer check stop condition*/
    pthread_mutex_unlock(&mtx);

    printf("[Producer] Finished work\n");
    pthread_exit(NULL);
}
/* in main (consumer) thread */
void* consumer(void* arg) {
    int units_consumed = 0;

    for (;;) {
        pthread_mutex_lock(&mtx);
        while (avail == 0 && !stop_producer ) {
            pthread_cond_wait(&cond, &mtx);
        }
```

```
        while (avail > 0) {                    /* Consume all available units */
            avail--;
            units_consumed++;
            printf("[Consumer] Consumed 1 unit. Exist: %d (Total consume: %d)\n", avail,
units_consumed);
        }
        /* Stop condition */
        if (stop_producer && avail == 0) {
                pthread_mutex_unlock(&mtx);
                break;
            }
        pthread_mutex_unlock(&mtx);
    }

    pthread_mutex_destroy(&mtx);

    printf("[Consumer]End. Total consumer: %d/%d\n", units_consumed, TOTAL_UNITS);

}
int main() {
    pthread_t prod_thread, cons_thread;
    printf("[Main] Start create Producer & Consumer\n");

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);

    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    pthread_mutex_destroy(&mtx);
    return 0;
}
```

Producer-Consumer: is a synchonize model in multi-threading, solve safely share data problem and efficiently run threads in parallel

- Producer: Create data, push it to Share Buffer/Queue
- Consumer: Pull data from Share Buffer/Queue

2.3 Semaphore

Syntax:

```
#include <semaphore.h>
int sem_init(sem_t *sem, int pshared, unsigned int value);
                                                        /* Returns 0 on success, or –1 on
error */
```

- **pshared**: indicates whether the sempaphore is to be shared between threads or processes
  - **pshared** is 0: semaphore is to be shared between the threads of the calling process --> If threads want to access, *sem* is typically specified as the address of either a local/static variable or is allocated on heap.
  - **pshared** is non-zero: shared between processes, *sem* must be the address of a location in a region of shared memory

Example:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define TOTAL_SLOTS 3
#define NUM_CARS 6

sem_t parking_slots;

void* car_thread(void* arg) {
    int car_id = *((int*) arg);
    printf("Car %d is coming and waiting for parking...\n", car_id);
    sem_wait(&parking_slots);

    printf("Car %d parked sucessfully\n", car_id);
    sleep(2);
    printf("Car %d left the parking lot\n", car_id);

    sem_post(&parking_slots);

    pthread_exit(NULL);
```

```
}
int main() {
    pthread_t cars[NUM_CARS];
    int car_ids[NUM_CARS];

    sem_init(&parking_slots, 0, TOTAL_SLOTS);

    printf("Open parking slot with %d empty spaces\n", TOTAL_SLOTS);

    for (int i = 0; i < NUM_CARS; i++) {
        car_ids[i] = i + 1;
        pthread_create(&cars[i], NULL, car_thread, &car_ids[i]);
    }

    for (int i = 0; i < NUM_CARS; i++) {
        pthread_join(cars[i], NULL);
    }

    sem_destroy(&parking_slots);
    printf("Finish");
    return 0;
}
```

Result:

```
Open parking slot with 3 empty spaces!
Car 2 is coming and waiting for parking...
Car 2 parked sucessfully
Car 1 is coming and waiting for parking...
Car 1 parked sucessfully
Car 3 is coming and waiting for parking...
Car 3 parked sucessfully                    /* We can see all 3 cars park at the moment,
MUTEX same */
Car 6 is coming and waiting for parking...
Car 4 is coming and waiting for parking...
Car 5 is coming and waiting for parking...
Car 2 left the parking lot
Car 3 left the parking lot
Car 1 left the parking lot      /* We can see maybe 2 or 3 cars left the parking lot, MUTEX
only 1 */
```

```
Car 4 parked sucessfully                /* Mutex will handle once, with 2s for each car */
Car 6 parked sucessfully
Car 5 parked sucessfully
Car 4 left the parking lot      /* Same, with the cars arriving*/
Car 6 left the parking lot
Car 5 left the parking lot
```

	Mutex	Semaphore
Meaning / Nature	Ownership lock	SIGNAL COUNTER
Share	Only 1 thread hold the lock at the moment	Allow N threads access
Unlock permission	Thread i lock --> thread i unlock	Any threads can call sem_post to increase Counter

2.4 Barrier

Defintion:

```
Thread 1 ──────────┐
Thread 2 ──────────┤
Thread 3 ──────────┤──→ BARRIER → all continue together
Thread 4 ──────────┘
```

--> Many threads wait together at the same time, then all can continue run

Example:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 3

pthread_barrier_t barrier;

void* worker(void *arg) {
    int id = *((int*) arg);

    printf("[Thread %d] is doing Phase 1...\n", id);
```

```
    sleep(id);
    printf("[Thread %d] finish Phase 1\n", id);

    pthread_barrier_wait(&barrier);   /*  threads must be go here, then continue */

    printf("[Thread %d] start Phase 2\n", id);

    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS] = {1, 2, 3};

    pthread_barrier_init(&barrier, NULL, NUM_THREADS);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_barrier_destroy(&barrier);
    return 0;
}
```

Result:

```
[Thread 2] is doing Phase 1...
[Thread 3] is doing Phase 1...
[Thread 1] is doing Phase 1...       /* wait here */
[Thread 1] finish Phase 1
[Thread 2] finish Phase 1
[Thread 3] finish Phase 1
[Thread 3] start Phase 2
[Thread 1] start Phase 2
[Thread 2] start Phase 2
```

3. Deadlock + Lock Internals + RWLock + Reentrancy

3.1 Deadlock

Definition:

```
Thread A                    Thread B
   |                           |
 lock(A)                     lock(B)
   |                           |
   |                         lock(A)
   |                           ↓
 lock(B) ← BLOCK             BLOCK
   |
   └──────── DEADLOCK ─────────┘
```

Lab 5

Example:

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

int global1 = 0;
int global2 = 100;

static pthread_mutex_t mtx1 = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t mtx2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1(void* arg) {
    pthread_mutex_lock(&mtx1);
    printf("Holding mtx1, waiting for mtx2 \n");
    sleep(2);
```

```
    pthread_mutex_lock(&mtx2);                                  /* Block here */

    global1++;
    global2++;

    pthread_mutex_unlock(&mtx2);
    pthread_mutex_unlock(&mtx1);

    pthread_exit(NULL);
}
void* thread2(void* arg) {
    pthread_mutex_lock(&mtx2);
    printf("Holding mtx2, waiting for mtx1 \n");
    sleep(2);

    pthread_mutex_lock(&mtx1);                                  /* Block here */

    global1++;
    global2++;

    pthread_mutex_unlock(&mtx1);
    pthread_mutex_unlock(&mtx2);

    pthread_exit(NULL);
}
int main() {
    pthread_t threads_1, threads_2;

    pthread_create(&threads_1, NULL, thread1, NULL);
    pthread_create(&threads_2, NULL, thread2, NULL);

    pthread_join(threads_1, NULL);
    pthread_join(threads_2, NULL);

    printf("Finish!");

    return 0;
}
```

Result:

```
Holding mtx1, waiting for mtx2
Holding mtx2, waiting for mtx1
.........              /* Block here since thread1 want to lock mtx2 and thread 2 want to
lock mtx1 */
```

Solution:

- **Mutex Hierachy**: Dev will regulate the *order* of Mutex Lock and **force** all threads follow that *order* (Ex: Completely locked *mtx1*, then go to lock *mtx2*
- **Try, and then back off** stragecy: use **pthread_mutex_lock()** for 1st mutex, but **pthread_mutex_trylock()** for next mutexs

4 necessary condtion occur Deadlock:

- Mutual exclusion: Only 1 thread use Resource at the moment (use Mutex)
- Hold and wait: Thread hold lock A but wait for lock B
- No preemption: Cant be automative get *lock* out of thread
- Circular wait:
  - T1 --> wait for T2
  - T2 --> wait for T1
  - T1 --> wait for T2

--> Deadlock will be occur if all 4 condition exist **Fix Deadlock**: We regulate that:

```
Always:
            lock(A)
              ↓
            lock(B)
```

```
Avoid:
            lock(B)
              ↓
            lock(A)
```

Example:

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

int global1 = 0;
```

```
static pthread_mutex_t mtx1 = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t mtx2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1(void* arg) {
    pthread_mutex_lock(&mtx1);
    printf("[Thread 1]Holding mtx1, waiting for mtx2\n");
    sleep(2);

    pthread_mutex_lock(&mtx2);

    global1++;

    printf("[Thread 1]Global 1 = %d\n", global1);

    pthread_mutex_unlock(&mtx2);
    pthread_mutex_unlock(&mtx1);

    printf("Finish Thread 1\n");
    pthread_exit(NULL);
}
void* thread2(void* arg) {
    pthread_mutex_lock(&mtx1);
    printf("[Thread 2] Holding mtx1, waiting for mtx2\n");
    sleep(2);

    pthread_mutex_lock(&mtx2);

    global1++;

    printf("[Thread 2]Global 1 = %d\n", global1);

    pthread_mutex_unlock(&mtx2);
    pthread_mutex_unlock(&mtx1);

    printf("[Thread 2]Finish Thread 2\n");

    pthread_exit(NULL);
}
int main() {
```

```
    pthread_t threads_1, threads_2;

    pthread_create(&threads_1, NULL, thread1, NULL);
    pthread_create(&threads_2, NULL, thread2, NULL);

    pthread_join(threads_1, NULL);
    pthread_join(threads_2, NULL);

    printf("Finish!");

    return 0;
}
```

So the result is:

```
[Thread 1]Holding mtx1, waiting for mtx2
[Thread 1]Global 1 = 1
Finish Thread 1
[Thread 2] Holding mtx1, waiting for mtx2
[Thread 2]Global 1 = 2
[Thread 2]Finish Thread 2
```

3.2 Lock Internals (Lock implementation concepts)

3.2.1 Atomic Instructions

Atomic Instruction is the command run by CPU which is **No Context Switch in-between** (none of thread or another CPU core can overwrite between read and write) Example (use atomic instruction):

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#include <stdatomic.h>                                  /* use this library */
```

```
// int counter = 0;
atomic_int counter = 0;                                          /* replace int to atomic_int */
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
void* worker(void* arg) {
    int thread_id = *((int*) arg);

    printf("[Thread %d] is running... (tid = %lu)\n", thread_id, pthread_self());
    for (int i = 0; i < 1000000; i++) {
        counter++;                                               /* no race condition here */
        // atomic_fetch_add(&counter, 1);
    }
    printf("[Thread %d] completely!\n", thread_id);

    pthread_exit(NULL);
}
int main() {
    pthread_t threads[4];
    int thread_ids[4] = {1, 2, 3, 4};

    printf("[Main] Start create 3 threads\n");

    for (int i = 0; i < 4; i++) {
        int ret = pthread_create(&threads[i], NULL, worker, (void*)&thread_ids[i]);
        printf("[Main] Thread %d is created\n", thread_ids[i]);
    }
    /*Join 4 threads together, wait for sucess of all 4 threads*/
    for (int i = 0;  i < 4; i++) {
        pthread_join(threads[i], NULL);
        printf("[Main] Joined Thread %d\n", thread_ids[i]);
    }
    printf("After 4 threads handle successfully, Counter = %d\n", counter);
    return 0;
}
```

Assembly code (hardware behind):

- Dont use Atomic:

```
mov eax, DWORD PTR counter[rip]    ; 1. Read from RAM to EAX register
add eax, 1                         ; 2. Increase 1 for EAX
mov DWORD PTR counter[rip], eax    ; 3. Write from EAX to RAM
```

- Use Atomic:

```
lock add DWORD PTR counter[rip], 1 ; /* Lock cache line and continue increase in RAM
*/
```

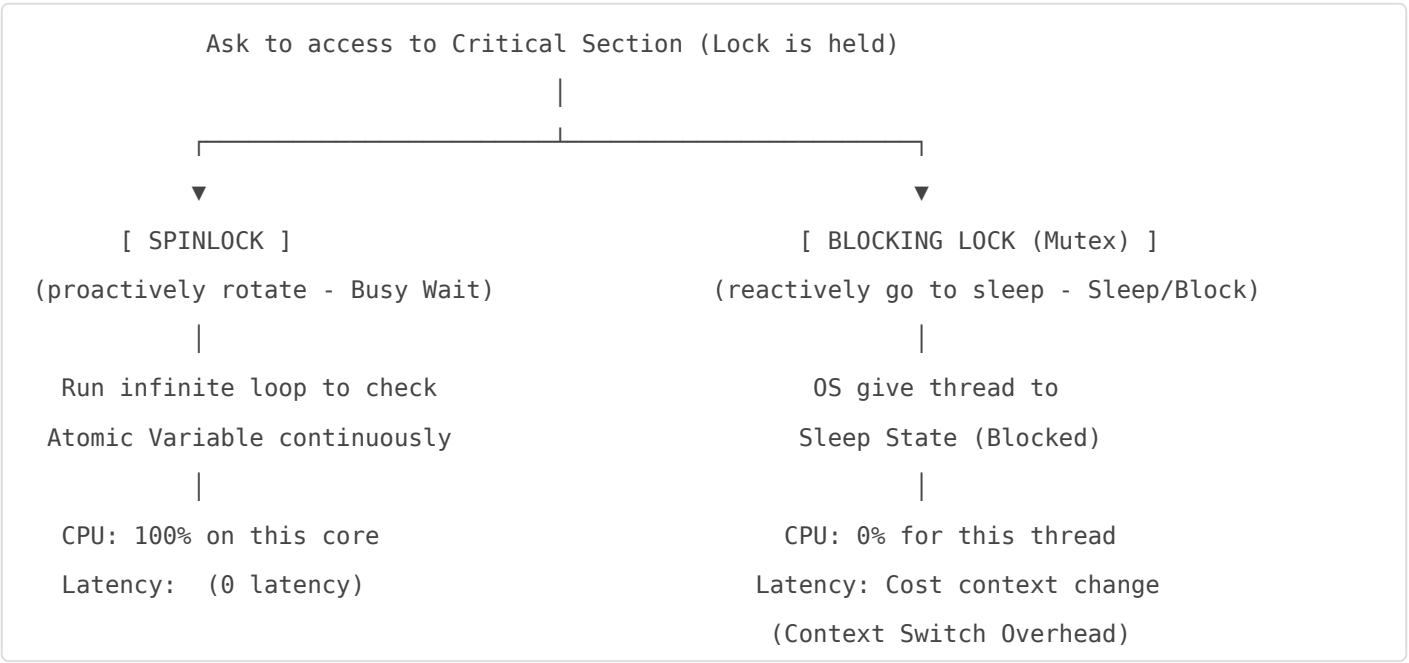
Compare:

Feature	Non-atomic (counter++)	Mutex Lock	Atomic Instruction
Mechanism	3 ways: LOAD -> ADD -> STORE	OS + synchonization software lock	Only 1 command lock bus/cache (LOCK INC, atomic_fetch_add)
Thread-safe	No (Race Condition)	Yes	Yes (Lock-free)
Overhead / Speed	Very fast (Wrong result)	Slower (cost of function call, change kernel context)	Very fast (directly handle at register/hardware)

3.2.2 Spin Lock & Blocking Lock

PROBLEM:
When one thread want to get the lock but this lock is **held** by another thread, should it wait (**Spin**) or sleep (**Block**)?

Definition:



SpinLock:
is built directly from Atomic Instruction. Spin lock **is programmed by only 1 Atomic function** is *atomic_flag_test_and_set()*

```
#include <stdatomic.h>

atomic_flag spinlock = ATOMIC_FLAG_INIT;

void spin_lock() {
    /* Read the flag and set 'true' in a single command to cannot be interrupted. */
    while (atomic_flag_test_and_set(&spinlock)) {
        /* rotate here */
    }
}

void spin_unlock() {
    atomic_flag_clear(&spinlock); // Unlock
}
```

Blocking lock
Mutex is a Blocking lock has the interference of OS
Whenever call *pthread_mutex_lock(&mtx)*:

- Check lock, if free --> get it (Atomic Instruction)
- If busy --> thread call **System Call** (`futex` on Linux) to ask OS Kernel: Let's take me to *Wait Queue*, close CPU and wake me up when unlock
- This cost Context Switch (save register, convert remember page table, change kernel state)

Compare:

Feature	Spinlock (pthread_spinlock_t)	Blocking Lock (pthread_mutex_t)
Mechanism (locked)	Continuously rotate CPU to verify	Change to Sleep State
CPU cost	Very high(100% on core spinning)	Near 0%
Context Switch Cost	0 (never leave CPU)	Yes (few micro-second)
Time to use lock efficiently	Very short time (few ns)	Long time (I/O, calculate a lot)"
The envirionment	Linux Kernel, hardware driver, Real-time	Nornal appliction (User-space application)

3.2.3 Futex

Futex is the machanism which combine between **Atomic Instruction** in User Space and **Wait Queue** of Kernel\

1. Fast Path (Uncontended - 90% time)
   - Handle in **User-space** by atomic command
   - Cost: few nano second, **0 System Call**, dont need Kernel
2. Slow Path (Contended - 10% time)
   - if lock is held, call **System Call** `futex(..., FUTEX_WAIT, ...)` then Kernal take it into **Wait Queue**
   - lock is free, see another thread is waiting, then call `futex(..., FUTEX_WAKE, ...)` to wake up by Kernel

2 basic function of futex:

- futex(addr, FUTEX_WAIT, val) : for sleep
- futex(addr, FUTEX_WAKE, num_threads): for wake up

--> We rarely call directly `futex()` function
--> Futex is **the base of all synchonization library** in Linux : pthread_mutex_t, pthread_cond_t, sem_t

3.3 Reader-Write Lock

With MUTEX, only 1 thread goes to **Critical Section**, although it only read data (no change anything)
--> Mutex doesn't know read or write (it thinks both are same) --> Waste the run in parallel --> **RWLock** efficiently fix it

2 type of RWLock:

1. Many reader can access at the same time
2. Only 1 writer can access, and whenever writer is accessing reader cannot access

NOTE: Reader and Writer cannot access at the same time

Example:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

int counter = 0;
pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;
```

```
void* reader(void *arg) {
    int id = *((int*) arg);
    for (int i = 0; i < 3; i++) {
        pthread_rwlock_rdlock(&rwlock);   /* many reader can access at the same time */
        printf("[Reader %d] counter = %d\n", id, counter);
        pthread_rwlock_unlock(&rwlock);
        usleep(500000);
    }
    return NULL;
}

void* writer(void *arg) {
    int id = *((int*) arg);
    for (int i = 0; i < 3; i++) {
        pthread_rwlock_wrlock(&rwlock);   /* only 1 writer */
        counter++;
        printf("[Writer %d] counter tang len = %d\n", id, counter);
        pthread_rwlock_unlock(&rwlock);
        usleep(500000);
    }
    return NULL;
}

int main() {
    pthread_t readers[3], writers[2];
    int rid[3] = {1,2,3}, wid[2] = {1,2};

    for (int i = 0; i < 3; i++) pthread_create(&readers[i], NULL, reader, &rid[i]);
    for (int i = 0; i < 2; i++) pthread_create(&writers[i], NULL, writer, &wid[i]);

    for (int i = 0; i < 3; i++) pthread_join(readers[i], NULL);
    for (int i = 0; i < 2; i++) pthread_join(writers[i], NULL);

    pthread_rwlock_destroy(&rwlock);
    return 0;
}
```

Result:

```
[Reader 1] counter = 0
[Reader 2] counter = 0
[Reader 3] counter = 0
[Writer 1] counter tang len = 1
[Writer 2] counter tang len = 2
[Reader 2] counter = 2
[Reader 3] counter = 2
[Reader 1] counter = 2
[Writer 1] counter tang len = 3
[Writer 2] counter tang len = 4
[Reader 2] counter = 4
[Reader 3] counter = 4
[Reader 1] counter = 4
[Writer 1] counter tang len = 5
[Writer 2] counter tang len = 6
```

3.4 Reentrancy & Thread_specific Data

3.4.1 Thread-safe and Reentrancy

A function is said to be **thread-safe** if it can safely be invoked by multiple threads at the same time

Conversely, if a function is **not thread-safe**, then we can’t call it from one thread while it is being executed in another thread
Cause: functions are use *shared variable* like **global** or **static** between threads

Reentrancy:
A reentrant function achieves thread safety **without the use of mutexes**. It does this by **avoiding** the use of **global and static variables**
Any information that must be returned to the caller, or maintained between calls to the function, is stored in **buffers allocated by the caller**

Lab 3

Example: (Non Reentrancy , Thread-safe)

```c
#include <stdio.h>
#include <pthread.h>
int generate_id(void) {
    static int last_id = 0;   /*  static variable : can be shared between threads */
    last_id++;
    return last_id;
}

void* worker(void *arg) {
    int local_ids[5];
    for (int i = 0; i < 5; i++) {
        local_ids[i] = generate_id();
    }
    for (int i = 0; i < 5; i++) {
        printf("[Thread %lu] got id = %d\n", pthread_self(), local_ids[i]);
    }
    return NULL;
}

int main() {
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, worker, NULL);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
    return 0;
}
```

--> Result: maybe occurs Duplicate ID or Lost Update (Race condition)

Fix it: (We have 2 ways)

1. Add mutex to protect `last_id` variable
2. Reentrancy : dont use static variable, state is transmited in/out by argument, each thread manage their variable Example (reentrancy):

```c
#include <stdio.h>
#include <pthread.h>
static pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
int generate_id_r(int *last_id) {
    (*last_id)++;
    return *last_id;
}
```

```
void* worker(void *arg) {
    int local_last_id = 0;   /* each thread has their variable , no share */
    for (int i = 0; i < 5; i++) {
        int id = generate_id_r(&local_last_id);
        printf("id = %d\n", id);
    }
    return NULL;
}
int main() {
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, worker, NULL);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
    return 0;
}
```

Result:

```
[Thread A] id = 1, 2, 3, 4, 5
[Thread B] id = 1, 2, 3, 4, 5
[Thread C] id = 1, 2, 3, 4, 5
[Thread D] id = 1, 2, 3, 4, 5
```

3.4.2 Thread-Specific Data

Definition:

```
Process
│
├── Thread A
│    └── TSD → state_A
│
├── Thread B
│    └── TSD → state_B
│
└── Thread C
     └── TSD → state_C
```

ROLE of TSD:
Thread-specific data is a technique for making an function unthread-safe to function thread-safe

without changing its interface
Thread-specific data allows a function to maintain a separate copy of a variable for each thread that calls the function

Thread-Specific Data API:

1. Key Creation: *pthread_key_create()* --> distinguish TSD memory from different functions
2. Destructor: after create key, we transmit destructor pointer
3. Allocate buffer memory for each thread
4. Link and access data: *pthread_setspecific(key, ptr)*

4. Threads + Signals + Fork + I/O + Integration

4.1 Threads and Signals

4.1.1. **Signal maps to Threads** (The ways Signal handle in multi-thread)

- **Process-wide**:
  - Signal action (default): unhandled signal whose default action is *stop* or *terminate* is delivered to any thread in a process, then **all of the threads** in the process are **stopped or terminated**
  - Signal dispositions: all threads in a process share the same disposition for each signal
- **Thread signal**:
  - Thread-directed signal: A signal is thread-directed if
    - hardware error when thread is running: SIGSEGV, SIGFPE, SIGILL, SIGBUS
    - SIGPIPE when thread try to write on a broken pipe/socket
    - is sent by *pthread_kill()* or *pthread_sigqueue()* from another thread in same process
  - Process-directed signal: Ctrl-C / Ctrl Z --> kernel will choose arbitrarily (random) thread is not avoiding this signal to execute handler
- **Per-thread**:
  - *Signal_mask*: per thread own a signal mask
  - Pending signals

- Alternate signal stack
- automatically start Pthread function

4.1.2 Manipulating the Thread Signal Mask__:

When a new thread is created, it inherits a copy of the signal mask of the thread that created (**pthread_create()**) it. A thread can use pthread_sigmask() to change its signal mask, to retrieve the existing mask, or both.

Syntax:

```
#include <signal.h>
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset);
                                        /* Returns 0 on success, or a positive error number on
error */
```

4.1.3 Sending a Signal to a Thread

The **pthread_kill()** function sends the signal sig to another thread in the same process. The target thread is identified by the argument thread. We cannot send it to another process

Syntax:

```
#include <signal.h>
int pthread_kill(pthread_t thread, int sig);
                                         /* Returns 0 on success, or a positive error number on
error */
```

or :

```
#define _GNU_SOURCE
#include <signal.h>
int pthread_sigqueue(pthread_t thread, int sig, const union sigval value);
                                       /* Returns 0 on success, or a positive error number on
error */
```

pthread_sigueue() allows send a signal with a small data (a interger or a pointer) to destination thread

4.1.4 Dealing with Asynchronous Signals Sanely

None of the functions in the Pthreads API is among the set of async-signal-safe functions that we can safely call from within a signal handler

2 ways to deal:

- block the signals in the **main thread** before any other threads are created --> child thread is created will inherit this mask
- Create a single dedicated thread that accepts incoming signals. This thread call sigwait(), sigwaitinfo(), sigtimedwait() to **wait synchronously signals** . Handle like a normal thread

4.2 Threads and fork()

When a multithreaded process calls fork(), only the calling thread is replicated in the child process All of the other threads vanish in the child; no thread-specific data destructors or cleanup handlers are executed for those threads

Example:

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/wait.h>

void* thread1(void* arg) {
    int thread_id = *((int*) arg);
    printf("[Thread %d] is running... (tid = %lu)\n",thread_id, pthread_self());
    pid_t pid = fork();
    if (pid == 0) {
        printf("[Thread %d][Child PID = %d]\n", thread_id, getpid());
        return 0;
    }
    if (pid > 0) {
        printf("[Thread %d][Parent pid = %d]\n",thread_id, getpid());
        wait(NULL);
    }
```

```
    pthread_exit(NULL);
}
void* thread2(void* arg) {

    int thread_id = *((int*) arg);
    printf("[Thread %d] is running... (tid = %lu)\n", thread_id,  pthread_self());
    pthread_exit(NULL);
}
void* thread3(void* arg) {

    int thread_id = *((int*) arg);
    printf("[Thread %d] is running... (tid = %lu)\n", thread_id,  pthread_self());
    pthread_exit(NULL);
}
int main() {
    pthread_t threads_1, threads_2,threads_3;
    int id1 = 1, id2 = 2, id3 = 3;

    pthread_create(&threads_1, NULL, thread1, (void*)&id1);
    pthread_create(&threads_2, NULL, thread2, (void*)&id2);
    pthread_create(&threads_3, NULL, thread3, (void*)&id3);

    pthread_join(threads_1, NULL);
    pthread_join(threads_2, NULL);
    pthread_join(threads_3, NULL);

    printf("PID of Main = %d\n", getpid());
    return 0;
}
```

Result:

```
[Thread 1] is running... (tid = 138357654943296)
[Thread 3] is running... (tid = 138357638157888)
[Thread 2] is running... (tid = 138357646550592)
[Thread 1][Parent pid = 77324]
[Thread 1][Child PID = 77328]
PID of Main = 77324
```

So we can see that after call `fork()` in thread 1 :

```
            Process P
           /          \
          /            \
      Parent P          Child C
       |                 |
       ├─ Thread 1       └─ Thread 1'
       ├─ Thread 2
       └─ Thread 3
```

Deadlock , Because Thread 1' is replicated (copy of memory) of Thread 1 --> Child inherit mutex state, but other threads is not created in Child

4.3 Threads + I/O

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>



static int pipe_fd[2];

static volatile int keep_running = 1;

void *io_thread(void *arg) {
    char buf[128];

    printf("[I/O Thread] Call read()...\n");

    ssize_t n = read(pipe_fd[0], buf, sizeof(buf) - 1);

    if (n > 0) {
        buf[n] = '\0';
        printf("[I/O Thread] Received data: \"%s\"\n", buf);
    }
```

```
    printf("[I/O Thread] finish, exit\n");
    return NULL;
}

void *worker_thread(void *arg) {
    int result = 0;

    printf("[Worker Thread] Start handle data (CPU-bound)...\n");

    for (int i = 0; i < 5; i++) {
        result += i * i;
        printf("[Worker Thread] handling... step %d, result = %d\n", i + 1, result);
        sleep(1);
    }

    printf("[Worker Thread] Finish, result = %d\n", result);
    keep_running = 0;   /* done */
    return NULL;
}

void *logger_thread(void *arg) {
    int tick = 0;

    while (keep_running) {
        printf("[Logger Thread] Still alive, tick = %d\n", tick++);
        sleep(1);
    }

    printf("[Logger Thread] See Worker finish, end\n");
    return NULL;
}

int main(void) {
    pthread_t t_io, t_worker, t_logger;

    if (pipe(pipe_fd) == -1) {
        perror("pipe");
        return 1;
    }
    pthread_create(&t_io, NULL, io_thread, NULL);
```

```
    pthread_create(&t_worker, NULL, worker_thread, NULL);
    pthread_create(&t_logger, NULL, logger_thread, NULL);



    sleep(4);
    printf("[Main] Send data to pipe\n");
    write(pipe_fd[1], "Hello from Main!", 17);

    pthread_join(t_io, NULL);
    pthread_join(t_worker, NULL);
    pthread_join(t_logger, NULL);

    close(pipe_fd[0]);
    close(pipe_fd[1]);

    printf("[Main] Finish\n");
    return 0;
}
```

Result:

```
[I/O Thread] Call read()...                         /* sleep here, wait for write */
[Worker Thread] Start handle data (CPU-bound)...
[Worker Thread] handling... step 1, result = 0
[Logger Thread] Still live, tick = 0
[Logger Thread] Still live, tick = 1
[Worker Thread] handling... step 2, result = 1
[Logger Thread] Still live, tick = 2
[Worker Thread] handling... step 3, result = 5
[Logger Thread] Still live, tick = 3
[Worker Thread] handling... step 4, result = 14
[Main] Send data to pipe                                  /* after 4s */
[I/O Thread] Received data: "Hello from Main!"
[I/O Thread] finish, exit
[Logger Thread] Still live, tick = 4
[Worker Thread] handling... step 5, result = 30
[Worker Thread] Finish, result = 30
[Logger Thread] See Worker finish, end
[Main] Finish
```

We can see:

- Blocking I/O only block self-thread call it
- I/O thread in wait state does not waste CPU but others thread allow use wait time for do another work

---

Revision #15
Created 2026-08-27 02:45:40 UTC by Lê Hoài Nam
Updated 2026-09-15 02:45:06 UTC by Lê Hoài Nam